KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 02

Sınıflar ve Nesneler

Yapıcı metotlar · this anahtar sözcüğü · Nesne yaşam döngüsü · Kapsüllemeye ilk adım

ÖZET

Geçen hafta ilk sınıfımızı yazdık ve `new` ile nesne ürettik; ancak nesnenin alanlarını **tek tek elle** doldurmak zorundaydık. Bu hafta o yaklaşımın neden kırılgan olduğunu göreceğiz ve çözümünü öğreneceğiz: **yapıcı metot** (constructor). Yapıcı, nesnenin doğduğu anda geçerli bir duruma sahip olmasını garanti eder. Ardından `this` anahtar sözcüğünü, yapıcı aşırı yüklemeyi, `this(...)` zincirini, nesnenin yaşam döngüsünü ve Java'da neden yıkıcı (destructor) bulunmadığını inceleyeceğiz.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası ... 3

1.1. Ön Koşul Bilgi Kontrolü ... 3
1.2. Haftanın Zaman Planı ... 4
1.3. Bu Haftanın Kavram Haritası ... 4

2. Motivasyon: Yarım Doğan Nesne ... 5

2.1. Geçen Haftanın Çözümü ... 5
2.2. Çıktının Söyledikleri ... 6
2.3. Sorunun Kökeni: Sorumluluğun Yanlış Yerde Olması ... 6

3. Yapıcı Metot: Tanım ve Sözdizimi ... 8

3.1. Yapıcının Üç Ayırt Edici Özelliği ... 8
3.2. Ogrenci Sınıfının Yapıcılı Sürümü ... 9
Satır satır çözümleme ... 9
3.3. Yapıcının Kullanımı ... 10
3.4. Bir Nesne Doğarken Ne Oluyor? ... 11

4. Derinleşme ... 13

4.1. this Anahtar Sözcüğü ve Alan Gölgeleme ... 13
4.2. Yapıcı Aşırı Yükleme ... 14
4.3. this(...) ile Yapıcı Zincirleme ... 17
4.4. Varsayılan Yapıcı ve Kaybolması ... 18
4.5. Kopya Yapıcı ve Referans Bağımsızlığı ... 20
4.6. toString() ile Anlamlı Metin Gösterimi ... 21
4.7. Nesne Yaşam Döngüsü, Çöp Toplayıcı ve Yıkıcı Sorunu ... 22
4.8. Kapsüllemeye İlk Adım: private Alan ve Denetimli Erişim ... 25

5. C'den Java'ya Köprü ... 28

6. Sık Yapılan Hatalar ... 29

6.1. Yapıcıya Dönüş Türü Yazmak ... 29
6.2. this'i Unutmak ... 29
6.3. Parametreli Yapıcı Ekleyip new Sinif() Kullanmaya Devam Etmek ... 30
6.4. this(...) Çağrısını İlk Deyim Yapmamak ... 30
6.5. Kopya Sanılan Atama ... 30
6.6. Derleyici İletileri Sözlüğü ... 31

7. İyi Tasarım İlkeleri ... 32

8. Bir Sonraki Haftaya Köprü ... 32

9. Haftanın Özeti ... 33

10. Alıştırmalar ... 35

10.1. Kavram Soruları ... 35
10.2. Kod Okuma ve Hata Bulma ... 35
10.3. Programlama Alıştırmaları ... 36

11. Öz Değerlendirme 38

Öz Değerlendirme Kontrol Listesi 38

12. Kaynaklar 38

12.1. Ders Kitapları 38

12.2. Çevrimiçi Kaynaklar 39

12.3. Bu Haftanın Kod Dosyaları 39

12.4. İletişim 39

1. Bu Haftanın Yol Haritası

Geçen hafta nesne yönelimli düşünmenin temel sorusunu sorduk: *"Bu problemde hangi varlıklar var, her biri neyi bilir ve ne yapabilir?"* `Ogrenci` sınıfını yazdık, `new Ogrenci()` ile nesne ürettik, alanlara değer atadık ve metotlarını çağırdık. Bu, doğru yönde atılmış ilk adımdı.

Ancak o çözümde sessiz bir kusur vardı. Nesne `new` ile doğduğu anda **boştu**: `ad` alanı `null`, `numara` alanı `0`, notlar `0.0` idi. Nesneyi kullanılabilir hâle getirmek, onu üreten kodun sorumluluğundaydı. Bir alanı atamayı unutmak veya geçersiz bir değer yazmak hiçbir uyarı üretmiyordu. Bu hafta tam olarak bu boşluğu kapatıyoruz.

Kapatma aracının adı **yapıcı metottur** (constructor). Yapıcı, bir nesnenin doğum anında çalışan ve ona geçerli bir başlangıç durumu veren özel bir kod bloğudur. C dilinde `struct` tanımladıktan sonra elle yazdığınız `hesap_ilklendir(&h, ...)` fonksiyonunun, dilin kendisi tarafından zorunlu kılınmış hâli olarak düşünebilirsiniz.

> **◎ Bu Bölümün Sonunda Öğrenci**
>
> - Yapıcı metodun ne olduğunu, sıradan bir metottan hangi üç noktada ayrıldığını (ad, dönüş türü, çağrılma biçimi) açıklayabilir.
> - Bir sınıfa **parametreli** ve **parametresiz** yapıcı yazabilir; nesneyi tek satırda geçerli durumda üretebilir.
> - **Yapıcı aşırı yüklemeyi** (constructor overloading) uygulayabilir ve derleyicinin hangi yapıcıyı hangi ölçüte göre seçtiğini açıklayabilir.
> - `this` anahtar sözcüğünü alan gölgelemesini çözmek, nesnenin kendisini döndürmek ve `this(...)` ile başka bir yapıcıyı çağırmak için kullanabilir.
> - **Varsayılan yapıcının** ne zaman var olduğunu, ne zaman kaybolduğunu ve kaybolduğunda derleyicinin verdiği hatayı yorumlayabilir.
> - **Kopya yapıcı** yazabilir ve iki referansın aynı nesneyi mi yoksa iki ayrı nesneyi mi gösterdiğini `==` ile sınayabilir.
> - Bir nesnenin **yaşam döngüsünü** (doğuş, kullanım, erişilemez hâle geliş, çöp toplama) betimleyebilir; Java'da yıkıcı bulunmamasının nedenini açıklayabilir.
> - `private` alan + `public` getter/setter düzenini kurarak **kapsüllemenin** ilk adımını atabilir ve doğrulama mantığını setter'a taşıyabilir.
> - `toString()` metodunu ezerek nesnenin anlamlı bir metin gösterimini üretebilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamını Hafta 1 bilgisiyle yanıtlayabiliyor olmalısınız. Yanıtlayamadığınız madde varsa, laboratuvar öncesinde Hafta 1 ders notunun ilgili bölümünü tekrar etmeniz önerilir.

1. Bir Java sınıfı ile o sınıftan üretilmiş bir nesne arasındaki fark nedir? Sınıf bellekte yer kaplar mı?
2. `new` anahtar sözcüğü tam olarak neyi yapar? Sonucunda ne döner?
3. `Ogrenci o1 = o2;` yazdığınızda kaç nesne olur? Neden?

4. Bir `int` alanı hiç atanmazsa değeri ne olur? Bir `String` alanı hiç atanmazsa?
5. Derleme zamanı hatası ile çalışma zamanı hatası arasındaki fark nedir? `NullPointerException` hangisidir?

1.2. Haftanın Zaman Planı

Süre	Etkinlik	İçerik
Teori 1. saat	Ders anlatımı	Yapıcısız nesnenin sorunu, yapıcı metodun tanımı ve sözdizimi, `this` (Bölüm 2 ve 3)
Teori 2. saat	Ders anlatımı + tartışma	Aşırı yükleme, `this(...)` zinciri, varsayılan yapıcı, kopya yapıcı, yaşam döngüsü, kapsüllemeye giriş (Bölüm 4, 5 ve 6)
Lab 1. saat	Rehberli uygulama	`Kitap` sınıfına yapıcı ekleme, aşırı yüklenmiş yapıcılar (Görev 0, 1 ve 2)
Lab 2. saat	Bireysel uygulama	`this` ile gölgeleme çözümü, hata ayıklama, banka hesabı bütünleştirme (Görev 3, 4 ve 5)

Tablo 2.1 — Haftalık 4 saatlik ders yükünün dağılımı

1.3. Bu Haftanın Kavram Haritası

Kavram	Ne işe yarar?	Hangi bölümde?
Yapıcı metot	Nesneyi doğduğu anda geçerli duruma getirir	Bölüm 3
`this` (referans)	Alan gölgelemesini çözer, nesnenin kendisini gösterir	Bölüm 4.1
Aşırı yükleme	Aynı sınıf için birden çok üretim yolu sunar	Bölüm 4.2
`this(...)` zinciri	Yapıcılar arası kod tekrarını yok eder	Bölüm 4.3
Varsayılan yapıcı	Hiç yapıcı yazmadığınızda derleyicinin ürettiği yapıcı	Bölüm 4.4
Kopya yapıcı	Var olan bir nesnenin bağımsız kopyasını üretir	Bölüm 4.5
`toString()`	Nesnenin okunabilir metin gösterimini verir	Bölüm 4.6
Yaşam döngüsü / GC	Nesnenin doğuşu ve belleğin geri alınması	Bölüm 4.7
`private` + getter/setter	Veriyi gizler, kuralı tek yerde toplar	Bölüm 4.8

Tablo 2.2 — Haftanın kavramları ve ele alındıkları bölümler

2. Motivasyon: Yarım Doğan Nesne

2.1. Geçen Haftanın Çözümü

Hafta 1'de `Ogrenci` sınıfını şöyle yazmıştık: alanlar `public`, yapıcı yok, nesne üretildikten sonra alanlar tek tek dolduruluyor. Aşağıdaki sınıf o sürümün birebir kopyasıdır; adını `OgrenciEski` yaparak bu haftanın yeni sürümüyle yan yana çalıştırabilelim diye sakladık.

OgrenciEski.java

```java
public class OgrenciEski {

    public String ad;
    public String soyad;
    public int numara;
    public double vizeNotu;
    public double finalNotu;

    public double ortalamaHesapla() {
        return vizeNotu * 0.4 + finalNotu * 0.6;
    }

    public void bilgiYaz() {
        System.out.printf("%-8d %-8s %-8s  ort: %6.2f%n",
                numara, ad, soyad, ortalamaHesapla());
    }
}
```

Kod 2.1 — Hafta 1'den kalan sürüm — hiç yapıcı yok

Bu sınıfla çalışan bir program yazalım. Amacımız sınıfı kötülemek değil; **nerede kırıldığını** görmek.

SorunGosterimi.java

```java
    public static void main(String[] args) {

        // --- Dogru kullanim: tum alanlar atanmis ---
        OgrenciEski o1 = new OgrenciEski();
        o1.ad = "Ali";
        o1.soyad = "Yilmaz";
        o1.numara = 331001;
        o1.vizeNotu = 65;
        o1.finalNotu = 78;
        o1.bilgiYaz();

        // --- SORUN 1: alanlar atanmadan nesne kullanilabiliyor ---
        OgrenciEski o2 = new OgrenciEski();
        System.out.print("Hicbir alani atanmamis nesne: ");
        o2.bilgiYaz();          // ad ve soyad null, notlar 0.0

        // --- SORUN 2: bir alan unutulursa hicbir uyari yok ---
        OgrenciEski o3 = new OgrenciEski();
        o3.ad = "Ayse";
        o3.soyad = "Demir";
        o3.numara = 331002;
        o3.vizeNotu = 70;
```

```
23          // o3.finalNotu = 85;   <-- UNUTULDU! Derleyici uyarmaz.
24          System.out.print("Final notu unutulmus nesne : ");
25          o3.bilgiYaz();          // ortalama yanlis hesaplanir
```

Kod 2.2 — Yapıcısız sınıfın üç sorunu

```
▸ TERMINAL — SorunGosterimi çıktısı
331001   Ali      Yilmaz    ort:  72.80
Hicbir alani atanmamis nesne: 0        null     null      ort:   0.00
Final notu unutulmus nesne : 331002   Ayse     Demir     ort:  28.00
Gecersiz degerli nesne     : -55      Mehmet   Kaya      ort:  76.00
```

2.2. Çıktının Söyledikleri

Program hiç hata vermeden çalıştı. Buna rağmen üç ayrı sorun üretti; hepsi de **sessiz** sorunlardır, yani derleyici ya da çalışma zamanı sizi uyarmaz.

#	Sorun	Çıktıdaki kanıt	Sonuç
1	Hiçbir alanı atanmamış nesne kullanılabiliyor	`0  null  null  ort: 0.00`	Nesne var ama anlamsız; `null` ile işlem yapılırsa `NullPointerException`
2	Bir alan atanması unutuluyor	`ort: 28.00` (olması gereken 79.00)	Hesap yanlış; program çöker değil, **yanlış sonuç** üretir — daha tehlikeli
3	Geçersiz değer engellenmiyor	`-55` numara, `250` ve `-40` not	Veri bütünlüğü kaybolur; hata çok sonra, başka bir yerde ortaya çıkar

Tablo 2.3 — Yapıcısız yaklaşımın ürettiği üç sessiz hata

Üçüncü satırı özellikle inceleyin. `o4.vizeNotu = 250;` satırı derlenir, çalışır ve hiçbir uyarı üretmez; çünkü `250` geçerli bir `double` değeridir. **Java'nın tür sistemi "bu bir sayıdır" der; "bu geçerli bir nottur" demez.** İş kuralını dile değil, sınıfa yazmak zorundayız.

> **▲ DİKKAT**
>
> Buradaki asıl tehlike programın çökmesi değil, **çökmemesidir**. Çöken program hatasını size gösterir. Sessizce yanlış sonuç üreten program, hatayı öğrenci not listesine, banka ekstresine veya hasta dosyasına taşır.

2.3. Sorunun Kökeni: Sorumluluğun Yanlış Yerde Olması

Yukarıdaki kodda nesnenin geçerli olmasından **nesneyi üreten kod** sorumludur. Bu sorumluluk yanlış yerdedir; çünkü:

- `OgrenciEski` sınıfı program içinde **onlarca yerde** üretilebilir; her yerin doğru doldurmayı hatırlaması gerekir.
- Sınıfa yeni bir alan eklendiğinde (örneğin `bolum`), o alanı atamayı unutan tüm kullanım yerleri sessizce eksik nesne üretmeye başlar.
- İş kuralı ("not 0–100 arasında olmalı") hiçbir yerde yazılı değildir; herkesin aklında tutması beklenir.

- Aynı doldurma kodu her kullanım yerinde yeniden yazılır — kopyala-yapıştır çoğalması ve bakım maliyeti.

Nesne yönelimli tasarımın temel ilkesi burada devreye girer: **Bir nesnenin geçerliliğinden o nesnenin sınıfı sorumludur.** Sorumluluğu sınıfın içine taşıyan araç, yapıcı metottur.



Şekil 2.1 — Yapıcısız ve yapıcılı yaklaşımın yan yana karşılaştırması. Solda nesne önce doğar sonra doldurulur; sağda nesne doğduğu anda eksiksizdir.

> **ⓘ BİLGİ**
>
> Bu ayrım yazılım mühendisliğinde **"geçersiz durum temsil edilemez olsun"** (make illegal states unrepresentable) ilkesiyle anılır. Amaç, hatayı yakalamak değil, hatanın oluşmasını **imkânsız** kılmaktır.

3. Yapıcı Metot: Tanım ve Sözdizimi

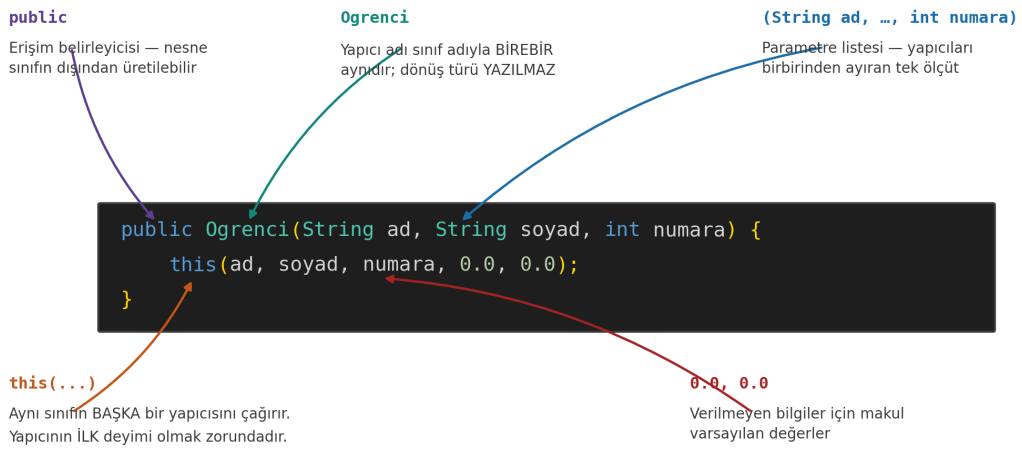
Yapıcı metot (constructor) `new` ile bir nesne üretilirken **otomatik olarak** çalışan, nesneye geçerli bir başlangıç durumu veren özel kod bloğudur. Adı sınıfın adıyla birebir aynıdır ve **dönüş türü yazılmaz**.

3.1. Yapıcının Üç Ayırt Edici Özelliği

Yapıcı, ilk bakışta sıradan bir metoda benzer; ama üç noktada ondan kesin biçimde ayrılır. Bu üç maddeyi ezberleyin, çünkü bu haftanın derleyici hatalarının neredeyse tamamı bunlardan birinin ihlalinden doğar.

Ölçüt	Yapıcı metot	Sıradan metot
Adı	Sınıf adıyla **birebir aynı** (`Ogrenci`)	İstenen herhangi bir ad (`ortalamaHesapla`)
Dönüş türü	**Yazılmaz.** `void` bile yazılmaz	Zorunlu: `void`, `int`, `String`, …
Çağrılma biçimi	Yalnızca `new` ile, nesne doğarken	Nesne üzerinden: `o.metot()`
Kaç kez çalışır?	Nesne başına tam **bir** kez	İstenildiği kadar çok kez
Kalıtılır mı?	Hayır (Hafta 7'de göreceğiz)	Evet
Aşırı yüklenebilir mi?	Evet	Evet

Tablo 2.4 — Yapıcı metot ile sıradan metot arasındaki farklar



Şekil 2.2 — Bir yapıcı metodun anatomisi. Dönüş türünün olmaması ve adın sınıf adıyla aynı olması zorunludur.

✖ SIK YAPILAN HATA

Yapıcının önüne yanlışlıkla `void` yazmak, bu haftanın **en sık** yapılan hatasıdır. `void Ogrenci(...)` yazdığınızda derleyici hata vermez; çünkü bu artık `Ogrenci` adında **sıradan bir metottur**. Sonuç: `new Ogrenci("Ali", ...)` satırı "cannot be applied to given types" hatası verir ve neden olduğunu anlamak zaman alır.

3.2. Ogrenci Sınıfının Yapıcılı Sürümü

Şimdi `Ogrenci` sınıfını yeniden yazalım. İki değişiklik yapıyoruz: alanları `private` yapıyoruz (Bölüm 4.8'de nedenini ayrıntılı göreceğiz) ve bir tam yapıcı ekliyoruz.

Ogrenci.java

```
1  public class Ogrenci {
2
3      // --- Ozellikler: artik private ---
4      private String ad;
5      private String soyad;
6      private int numara;
7      private double vizeNotu;
8      private double finalNotu;
9
10     /**
11      * Tam yapici: tum bilgileri alir ve dogrular.
12      * Nesne olusturulur olusturulmaz gecerli bir duruma sahip olur.
13      */
14     public Ogrenci(String ad, String soyad, int numara,
15                    double vizeNotu, double finalNotu) {
16         this.ad = ad;
17         this.soyad = soyad;
18         this.numara = numara;
19         this.vizeNotu = notuDuzelt(vizeNotu);
20         this.finalNotu = notuDuzelt(finalNotu);
21     }
```

Kod 2.3 — Alanlar artık private; tam yapıcı bilgileri alıp doğruluyor

Satır satır çözümleme

- **Satır 11–15:** Alanlar `private` oldu. Artık `o.vizeNotu = 250;` satırı sınıfın dışından **derlenmez**. Sorun 3 kaynağında yok edildi.
- **Satır 21–22:** Yapıcının imzası. `public` erişim belirleyicisi, `Ogrenci` sınıf adı, ardından parametre listesi. Arada dönüş türü **yoktur**.
- **Satır 23–25:** `this.ad = ad;` — parametre ile alan aynı adı taşıdığı için `this` zorunludur. Bunu Bölüm 4.1'de ayrıntılı inceleyeceğiz.
- **Satır 26–27:** Notlar doğrudan atanmıyor; `notuDuzelt()` süzgecinden geçiriliyor. İş kuralı artık sınıfın içinde, tek yerde tanımlı.

Doğrulama mantığını yapan yardımcı metot şudur:

Ogrenci.java

```
1      /** Not 0-100 araliginda degilse sinira cekilir. */
2      private static double notuDuzelt(double not) {
3          if (not < 0) {
4              return 0.0;
5          }
6          if (not > 100) {
7              return 100.0;
8          }
9          return not;
10     }
```

Kod 2.4 — Doğrulama tek bir özel (private) yardımcı metotta toplanmış

> **İPUCU**
>
> `notuDuzelt` metodu hem `private` hem `static`'tir. `private` çünkü sınıfın dışını ilgilendirmez; `static` çünkü herhangi bir nesnenin durumuna değil, yalnızca kendisine verilen sayıya bakar. `static`'i Hafta 6'da ayrıntılı göreceğiz.

3.3. Yapıcının Kullanımı

OgrenciDemo.java

```
    public static void main(String[] args) {

        // Tam yapici: nesne tek satirda ve gecerli durumda dogar
        Ogrenci o1 = new Ogrenci("Ali", "Yilmaz", 331001, 65, 78);

        // Kisa yapici: notlar sonra girilecek
        Ogrenci o2 = new Ogrenci("Ayse", "Demir", 331002);

        // toString() sayesinde nesne dogrudan yazdirilabiliyor
        System.out.println("--- Baslangic ---");
        System.out.println(o1);
        System.out.println(o2);

        // Notlar sonradan, denetimli bicimde guncelleniyor
        o2.notlariGuncelle(40, 52);
        System.out.println("\n--- Ayse'nin notlari girildi ---");
        System.out.println(o2);

        // Gecersiz degerler yapici tarafindan duzeltiliyor
        Ogrenci o3 = new Ogrenci("Mehmet", "Kaya", 331003, 250, -40);
        System.out.println("\n--- Gecersiz not denemesi (250 ve -40) ---");
        System.out.println(o3);
```

Kod 2.5 — Yapıcılı sınıfın kullanımı

```
▸ TERMINAL — OgrenciDemo çıktısı (ilk bölüm)
--- Baslangic ---
331001 | Ali Yilmaz     | vize: 65.0 final: 78.0 ort: 72.80 | GECTI
331002 | Ayse Demir     | vize:  0.0 final:  0.0 ort:  0.00 | KALDI

--- Ayse'nin notlari girildi ---
331002 | Ayse Demir     | vize: 40.0 final: 52.0 ort: 47.20 | KALDI

--- Gecersiz not denemesi (250 ve -40) ---
331003 | Mehmet Kaya    | vize:100.0 final:  0.0 ort: 40.00 | KALDI

--- Kopya yapici deneyi ---
```

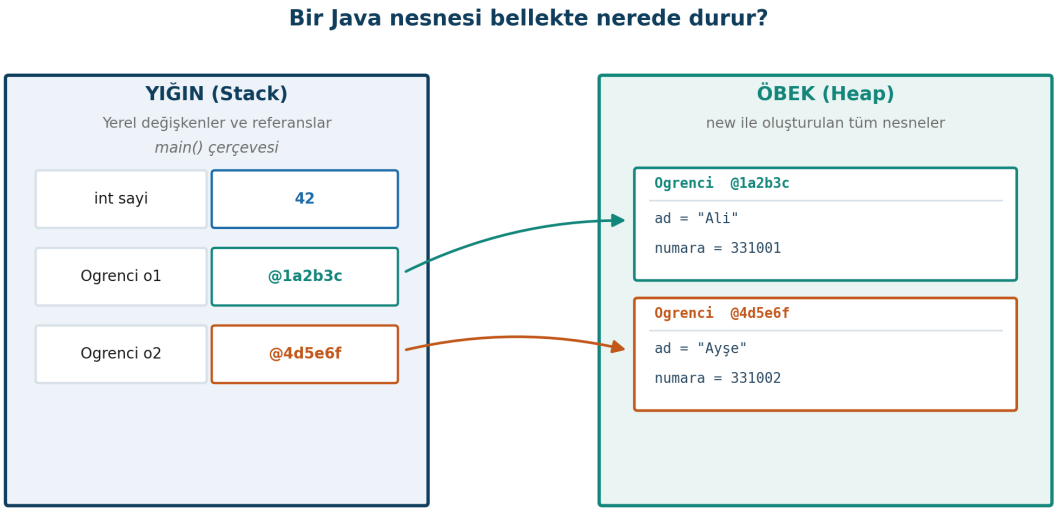
Çıktıdaki üçüncü bloğa dikkat edin: `new Ogrenci("Mehmet", "Kaya", 331003, 250, -40)` çağrısında verilen geçersiz notlar, nesneye ulaşmadan önce `notuDuzelt` tarafından `100.0` ve `0.0` değerlerine çekilmiştir. Aynı deneme Bölüm 2'deki yapıcısız sürümde hiçbir engelle karşılaşmamıştı.

> **ⓘ BİLGİ**
>
> `System.out.println(o1)` çağrısında `o1` bir nesnedir, metin değildir. Java bu durumda otomatik olarak nesnenin `toString()` metodunu çağırır. Biz bu metodu `Ogrenci` sınıfında ezdiğimiz için okunabilir bir satır görüyoruz. Ayrıntısı Bölüm 4.6'da.

3.4. Bir Nesne Doğarken Ne Oluyor?

`new Ogrenci("Ali", "Yilmaz", 331001, 65, 78)` ifadesi tek bir işlem gibi görünür; aslında art arda dört adım gerçekleşir:

1. **Bellek ayrılır.** JVM öbekte (heap) `Ogrenci` nesnesinin alanlarını tutacak kadar yer açar.
2. **Alanlar varsayılan değerlere kurulur.** Sayısal alanlar `0`, `boolean` alanlar `false`, referans alanlar `null` olur. Bu, yapıcı çalışmadan **önce** olur.
3. **Yapıcı çalışır.** `this(...)` zinciri varsa önce o işler; ardından gövde yürütülür ve alanlar gerçek değerlerini alır.
4. **Referans döner.** `new` ifadesinin değeri, öbekteki nesnenin adresidir; bu adres `o1` değişkenine atanır.



Şekil 2.3 — Hafta 1'den hatırlatma: değişken nesnenin kendisini değil, öbekteki nesnenin adresini tutar.

İkinci adım, sıkça gözden kaçan ama sınavda sorulan bir ayrıntıdır: **alanlar yapıcı çalışmadan önce zaten varsayılan değerlerine kurulmuştur.** Bu nedenle yapıcıda atamadığınız bir alan `null` veya `0` kalır; "tanımsız çöp değer" içermez. C'deki `malloc` ile ayrılmış belleğin rastgele içerik taşımasından farklıdır.

Alan türü	Varsayılan değer	Gerçek çalıştırma çıktısı
`int`, `long`, `short`, `byte`	`0`	`int -> 0`

Alan türü	Varsayılan değer	Gerçek çalıştırma çıktısı
`double`, `float`	`0.0`	`double -> 0.0`
`boolean`	`false`	`boolean -> false`
`char`	`'\u0000'` (boş karakter)	`char -> \u0000 (bos karakter)`
`String` ve tüm referans türleri	`null`	`String -> null`
Diziler (`int[]`)	`null` (dizi nesnesi henüz yok)	`int[] -> null`

Tablo 2.5 — Alanların yapıcı çalışmadan önceki varsayılan değerleri (VarsayilanYapici.java çıktısından)

```
▸ TERMINAL — VarsayilanYapici çıktısı — varsayılan değerler
Alanlarin varsayilan degerleri:
  int     -> 0
  double  -> 0.0
  boolean -> false
  char    -> \u0000 (bos karakter)
  String  -> null
  int[]   -> null
```

▲ DİKKAT

Bu tablo yalnızca **alanlar** için geçerlidir. Metot içindeki **yerel değişkenler** varsayılan değer almaz; atanmadan kullanılırlarsa derleyici `variable x might not have been initialized` hatası verir. Bu, Java'nın C'ye göre önemli bir güvenlik kazanımıdır.

4. Derinleşme

4.1. this Anahtar Sözcüğü ve Alan Gölgeleme

this Metodun veya yapıcının **üzerinde çalıştığı nesnenin kendisine** verilen örtük referanstır. Her nesne metodunun içinde kullanılabilir; `static` metotlarda kullanılamaz, çünkü orada ortada bir nesne yoktur.

Yapıcı parametrelerine alanlarla aynı adı vermek Java'da yerleşik bir alışkanlıktır ve okunabilirliği artırır. Ancak bu durumda metot gövdesinde `ad` yazdığınızda derleyici **parametreyi** anlar; alan **gölgelenir** (shadowing). Alana ulaşmanın tek yolu `this.` ön ekidir.

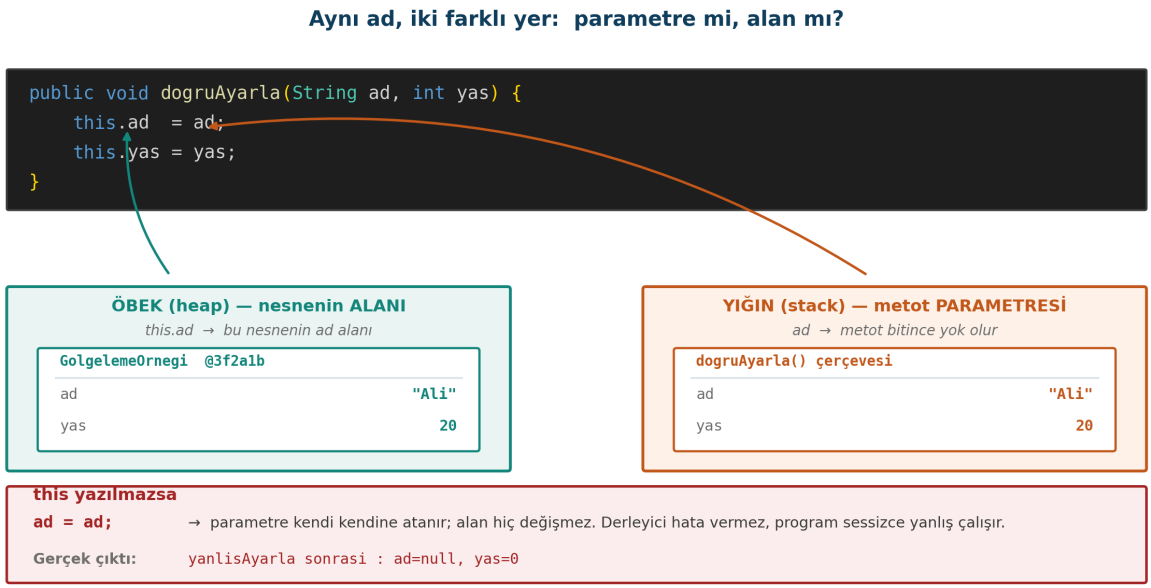
GolgelemeOrnegi.java

```java
/** YANLIS yaklasim: this yok, atama kendine yapiliyor. */
public void yanlisAyarla(String ad, int yas) {
    ad = ad;        // parametre kendine atanir; alan degismez!
    yas = yas;      // ayni sorun
}

/** DOGRU yaklasim: this ile alan acikca belirtiliyor. */
public void dogruAyarla(String ad, int yas) {
    this.ad = ad;
    this.yas = yas;
}

/** Alternatif: parametre adini farklilastirmak. */
public void farkliAdlaAyarla(String yeniAd, int yeniYas) {
    ad = yeniAd;    // golgeleme yok, this gerekmez
    yas = yeniYas;
}
```

Kod 2.6 — Aynı sorunun üç farklı ele alınışı: yanlış, doğru ve kaçınmalı



Şekil 2.4 — `this.ad` öbekteki alanı, çıplak `ad` ise yığındaki parametreyi gösterir. `this` yazılmazsa parametre kendi kendine atanır.

`yanlisAyarla` metodundaki `ad = ad;` satırı **derlenir**; hiçbir uyarı vermez. Yaptığı iş parametreyi kendisine atamaktır: hiçbir şey. Alan `null` kalır. Gerçek çıktı bunu açıkça gösterir:

```
▸ TERMINAL — GolgelemeOrnegi çıktısı
yanlisAyarla sonrasi : ad=null, yas=0
dogruAyarla sonrasi  : ad=Ali, yas=20
farkliAdla sonrasi   : ad=Ayse, yas=22
uc kez yasiArtir()   : ad=Ayse, yas=25
```

Kullanım	Anlamı	Ne zaman gerekir?
`this.alan = parametre;`	Bu nesnenin alanına ata	Parametre adı alan adıyla aynıysa **zorunlu**
`this.metot();`	Bu nesnenin metodunu çağır	Genellikle gereksiz; okunabilirlik için tercih edilebilir
`return this;`	Nesnenin kendisini döndür	Zincirleme çağrı (`a.x().y().z()`) kurmak için
`this(...)`	Aynı sınıfın başka yapıcısını çağır	Yapıcılar arası kod tekrarını önlemek için (Bölüm 4.3)
`baskaMetot(this)`	Kendini başka bir metoda parametre olarak ver	Nesnenin kendisini bir listeye/işleyiciye kaydettirirken

Tablo 2.6 — `this` anahtar sözcüğünün dört kullanım biçimi

`return this;` kullanımı, aşağıdaki gibi **akıcı** (fluent) çağrılar yazmayı mümkün kılar. Laboratuvar Görev 3'te `Zaman` sınıfında bunu uygulayacaksınız.

GolgelemeOrnegi.java

```java
/** this, nesnenin kendisini dondurebilir (zincirleme kullanim). */
public GolgelemeOrnegi yasiArtir() {
    this.yas++;
    return this;
}
```

Kod 2.7 — `return this` ile zincirleme çağrı

> **İPUCU**
>
> Bazı kaynaklar "gölgelemeden kaçının, parametre adına `yeniAd` deyin" der. Modern Java uygulamalarında yaygın olan yaklaşım tersidir: **parametreye alanla aynı adı verin ve `this` kullanın.** Böylece parametre listesi okunduğunda hangi alanın doldurulduğu doğrudan anlaşılır.

4.2. Yapıcı Aşırı Yükleme

Aşırı yükleme (overloading) Aynı sınıfta, aynı ada sahip fakat **parametre listesi farklı** birden çok yapıcı veya metot bulunmasıdır. Derleyici, çağrıdaki argümanların sayısına, türüne ve sırasına bakarak hangisinin çalışacağına **derleme zamanında** karar verir.

Bir sınıfın nesnesi her zaman aynı miktarda bilgiyle üretilemez. Notu henüz girilmemiş bir öğrenci, fiyatı bilinmeyen bir kitap, yalnızca köşegeni bilinen bir nokta… Her senaryo için ayrı bir yapıcı yazmak, çağrı yerini hem kısaltır hem de niyeti açık hâle getirir.

Nokta.java

```java
public class Nokta {

    private double x;
    private double y;

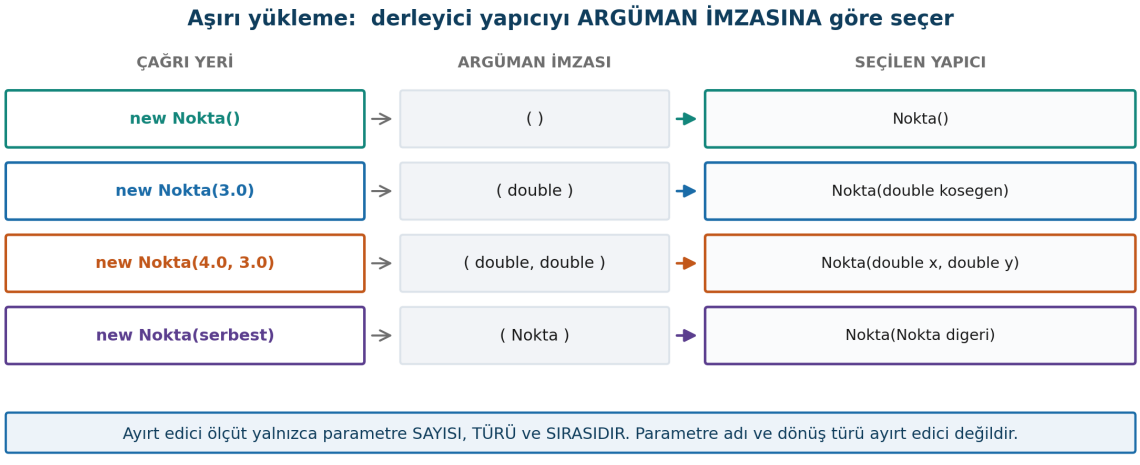
    /** Parametresiz yapici: orijin noktasi. */
    public Nokta() {
        this(0.0, 0.0);          // tam yapiciyi cagirir
    }

    /** Tek parametreli: kosegen uzerindeki nokta (x = y). */
    public Nokta(double kosegen) {
        this(kosegen, kosegen);
    }

    /** Tam yapici: gercek isi yapan tek yapici budur. */
    public Nokta(double x, double y) {
        this.x = x;
        this.y = y;
    }

    /** Kopya yapici. */
    public Nokta(Nokta digeri) {
        this(digeri.x, digeri.y);
    }
```

Kod 2.8 — Dört farklı yapıcı, tek bir sınıf



Şekil 2.5 — Derleyici, çağrıdaki argüman imzasına bakarak yapıcıyı seçer. Parametre adı ve dönüş türü ayırt edici değildir.

İmza çifti	Geçerli mi?	Neden?
`Nokta()` / `Nokta(double)`	**Geçerli**	Parametre sayısı farklı
`Nokta(double)` / `Nokta(int)`	**Geçerli**	Parametre türü farklı
`Nokta(double, int)` / `Nokta(int, double)`	**Geçerli**	Parametre sırası farklı

İmza çifti	Geçerli mi?	Neden?
`Nokta(double x)` / `Nokta(double kosegen)`	**Geçersiz**	Yalnızca parametre **adı** farklı — imza aynı
`Nokta(Nokta)` / `Nokta(Ogrenci)`	**Geçerli**	Parametre türü farklı

Tablo 2.7 — Aşırı yüklemede geçerli ve geçersiz imza çiftleri

> **▲ DİKKAT**
>
> Dönüş türü aşırı yüklemede **ayırt edici değildir**. Bu kural sıradan metotlar için de geçerlidir: `int hesapla(int a)` ile `double hesapla(int a)` aynı sınıfta bulunamaz. Yapıcılarda zaten dönüş türü olmadığı için bu soru hiç doğmaz.

Aşırı yüklenmiş yapıcıların kullanımı ve kopya yapıcının bağımsızlık sınaması:

NoktaDemo.java

```
1       public static void main(String[] args) {
2           Nokta orijin = new Nokta();            // parametresiz
3           Nokta kosegen = new Nokta(3.0);        // tek parametreli
4           Nokta serbest = new Nokta(4.0, 3.0);   // iki parametreli
5           Nokta kopya = new Nokta(serbest);      // kopya yapici
6
7           System.out.println("orijin  = " + orijin);
8           System.out.println("kosegen = " + kosegen);
9           System.out.println("serbest = " + serbest);
10          System.out.println("kopya   = " + kopya);
11
12          System.out.printf("%nserbest'in orijine uzakligi: %.2f%n",
13                  serbest.orijineUzaklik());
14          System.out.printf("kosegen ile serbest arasi   : %.2f%n",
15                  kosegen.uzaklik(serbest));
16
17          // Kopya gercekten bagimsiz mi?
18          kopya.tasi(10, 10);
19          System.out.println("\n--- kopya.tasi(10, 10) sonrasi ---");
20          System.out.println("serbest = " + serbest);   // degismedi
21          System.out.println("kopya   = " + kopya);
22          System.out.println("serbest == kopya -> " + (serbest == kopya));
```

Kod 2.9 — Dört yapıcının kullanımı ve kopyanın bağımsızlık sınaması

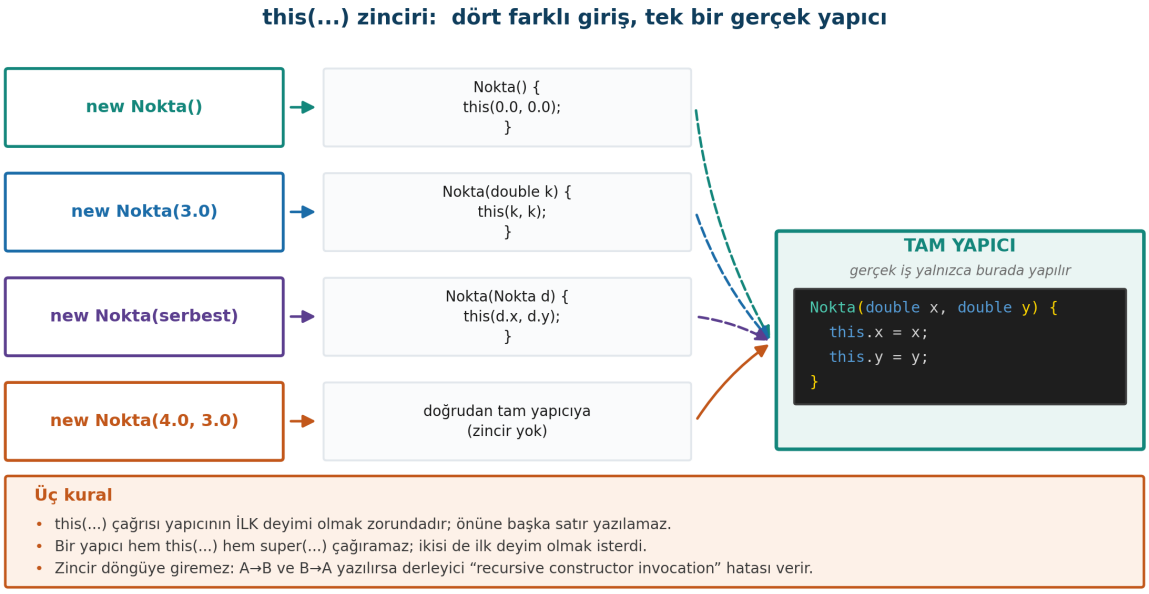
```
▸ TERMINAL — NoktaDemo çıktısı
orijin  = (0.00, 0.00)
kosegen = (3.00, 3.00)
serbest = (4.00, 3.00)
kopya   = (4.00, 3.00)

serbest'in orijine uzakligi: 5.00
kosegen ile serbest arasi   : 1.00

--- kopya.tasi(10, 10) sonrasi ---
serbest = (4.00, 3.00)
kopya   = (14.00, 13.00)
serbest == kopya -> false
```

4.3. this(...) ile Yapıcı Zincirleme

`Nokta` sınıfının dört yapıcısına yeniden bakın. Yalnızca biri gerçek iş yapar (`this.x = x; this.y = y;`); diğer üçü `this(...)` ile ona yönlendirir. Bu desene **yapıcı zincirleme** (constructor chaining) denir ve amacı kod tekrarını sıfıra indirmektir.



Şekil 2.6 — Dört farklı giriş noktası, tek bir gerçek yapıcıya yönlendirilir. Doğrulama mantığı tek yerde bulunur.

Zincirlemenin somut kazancı şudur: `Nokta` sınıfına ileride bir doğrulama kuralı ("koordinatlar `NaN` olamaz") eklemek isterseniz, o kuralı **yalnızca tam yapıcıya** yazarsınız. Diğer üç yapıcı zaten oradan geçtiği için kural otomatik olarak hepsinde geçerli olur.

Ogrenci.java

```java
/**
 * Kisa yapici: notlar henuz girilmemis ogrenciler icin.
 * this(...) ile tam yapiciyi cagirir; kod tekrari olmaz.
 */
public Ogrenci(String ad, String soyad, int numara) {
    this(ad, soyad, numara, 0.0, 0.0);
}

/**
 * Kopya yapici: var olan bir ogrencinin bagimsiz kopyasini uretir.
 */
public Ogrenci(Ogrenci kaynak) {
    this(kaynak.ad, kaynak.soyad, kaynak.numara,
         kaynak.vizeNotu, kaynak.finalNotu);
}
```

Kod 2.10 — `Ogrenci` sınıfında zincirleme: kısa yapıcı ve kopya yapıcı

Kural	Açıklama	İhlal edilirse
`this(...)` ilk deyim olmalı	Yapıcının ilk satırı olmak zorundadır; önüne başka deyim yazılamaz	`call to this must be first statement in constructor`

Kural	Açıklama	İhlal edilirse
Bir yapıcıda tek çağrı	`this(...)` ve `super(...)` aynı yapıcıda birlikte bulunamaz	Derleme hatası (ikisi de ilk deyim olmak ister)
Zincir döngüsel olamaz	A yapıcısı B'yi, B de A'yı çağıramaz	`recursive constructor invocation`
Zincir sonlu olmalı	Zincirin ucunda gerçek işi yapan bir yapıcı bulunmalıdır	Sonsuz döngü derleyicide yakalanır

Tablo 2.8 — `this(...)` zincirinin dört kuralı

4.4. Varsayılan Yapıcı ve Kaybolması

Hafta 1'de hiç yapıcı yazmadık, buna rağmen `new Ogrenci()` çalıştı. Nasıl? Çünkü **hiçbir yapıcı yazmadığınızda derleyici sizin için parametresiz, boş gövdeli bir yapıcı üretir.** Buna *varsayılan yapıcı* (default constructor) denir. Ancak kendi yapıcınızı yazdığınız anda derleyici bu üretimden vazgeçer.

VarsayilanYapici.java

```java
// ---- Durum 1: hic yapici yok -> derleyici varsayilani uretir ----
static class A {
    int deger;
    // Derleyici gizlice sunu ekler:  A() { }
}

// ---- Durum 2: parametreli yapici var -> varsayilan KAYBOLUR ----
static class B {
    int deger;

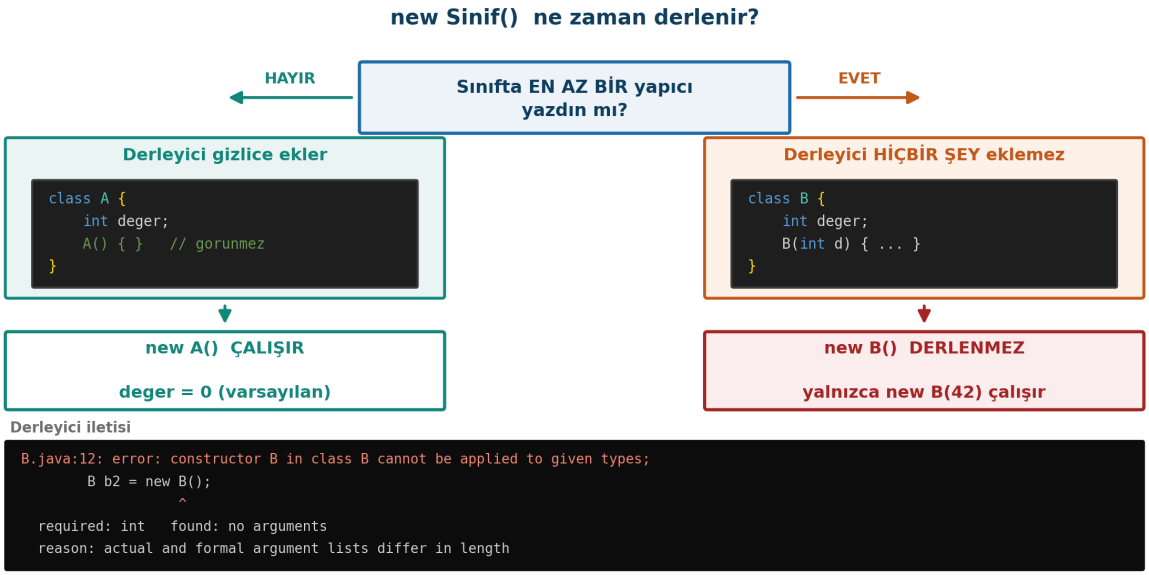
    B(int deger) {
        this.deger = deger;
    }
    // new B();  <-- DERLENMEZ: parametresiz yapici artik yok
}

// ---- Durum 3: ikisini de acikca yaziyoruz -> ikisi de kullanilabilir ----
static class C {
    int deger;

    C() {
        this(0);
    }

    C(int deger) {
        this.deger = deger;
    }
}
```

Kod 2.11 — Üç durum: hiç yapıcı yok, yalnızca parametreli var, ikisi de var



Şekil 2.7 — Varsayılan yapıcının var olup olmadığını belirleyen tek soru ve derleyicinin verdiği hata iletisi.

Sınıfta yazılı yapıcı	Derleyici ne yapar?	`new Sinif()`	`new Sinif(42)`
Hiç yok	`Sinif() { }` üretir	**Çalışır**	Derlenmez
Yalnızca `Sinif(int)`	Hiçbir şey üretmez	**Derlenmez**	Çalışır
`Sinif()` ve `Sinif(int)`	Hiçbir şey üretmez	Çalışır	Çalışır
Yalnızca `private Sinif()`	Hiçbir şey üretmez	Sınıf dışından derlenmez	Derlenmez

Tablo 2.9 — Varsayılan yapıcının dört senaryodaki durumu

```
▸ TERMINAL — VarsayilanYapici çıktısı (A, B, C)
A varsayilan yapici ile olustu, deger = 0
B parametreli yapici ile olustu, deger = 42
C: c1.deger = 0, c2.deger = 7
```

✖ SIK YAPILAN HATA

Bu, dönemin en sık karşılaşılan derleme hatalarından biridir. Bir sınıfa parametreli yapıcı eklediğiniz anda, o sınıfı `new Sinif()` biçiminde üreten **eski kodlarınız derlenmez olur**. Çözüm: parametresiz yapıcıya hâlâ ihtiyacınız varsa onu **açıkça yazın** — tercihen `this(varsayilan, varsayilan)` biçiminde zincirleyerek.

ⓘ BİLGİ

Dördüncü satır ileride işinize yarayacak: yapıcıyı `private` yapmak, sınıfın dışarıdan `new` ile üretilmesini engeller. Bu, **Singleton** ve **statik fabrika metodu** desenlerinin temelidir; Hafta 14'te göreceğiz.

4.5. Kopya Yapıcı ve Referans Bağımsızlığı

Kopya yapıcı Kendi sınıfından bir nesneyi parametre alan ve onun alan değerlerini yeni nesneye taşıyan yapıcıdır. Sonuç, kaynaktan **bağımsız** yeni bir nesnedir.

Hafta 1'de `Ogrenci o2 = o1;` yazdığımızda **iki değişkenin tek nesneyi** gösterdiğini görmüştük. Çoğu zaman istenen budur; ancak gerçekten ayrı bir kopya gerektiğinde kopya yapıcı kullanılır.

OgrenciDemo.java

```
        // Kopya yapici: bagimsiz bir kopya
        Ogrenci kopya = new Ogrenci(o1);
        kopya.notlariGuncelle(95, 95);
        System.out.println("\n--- Kopya yapici deneyi ---");
        System.out.println("Ozgun : " + o1);
        System.out.println("Kopya : " + kopya);
        System.out.println("o1 == kopya -> " + (o1 == kopya));

        // Nesne dizisi
        Ogrenci[] sinif = { o1, o2, o3 };
        double toplam = 0;
        for (Ogrenci o : sinif) {
            toplam += o.ortalamaHesapla();
        }
        System.out.printf("%nSinif ortalamasi: %.2f%n", toplam / sinif.length);
    }
```

Kod 2.12 — Kopya yapıcının bağımsızlık sınaması ve nesne dizisi

```
▸ TERMINAL — OgrenciDemo çıktısı (kopya deneyi)
Kopya : 331001 | Ali Yilmaz     | vize: 95.0 final: 95.0 ort: 95.00 | GECTI
o1 == kopya -> false

Sinif ortalamasi: 53.33
```

İfade	Kaç nesne?	`o1 == o2` sonucu	Birini değiştirmek diğerini etkiler mi?
`Ogrenci o2 = o1;`	1	`true`	**Evet** — aynı nesne
`Ogrenci o2 = new Ogrenci(o1);`	2	`false`	Hayır — bağımsız kopya
`Ogrenci o2 = new Ogrenci("Ali", …);`	2	`false`	Hayır

Tablo 2.10 — Atama, kopya yapıcı ve yeni üretim arasındaki fark

> **▲ DİKKAT**
>
> `==` işleci referans türlerinde **adres karşılaştırır**, içerik değil. İçerik karşılaştırması için `equals()` metodu kullanılır; onu Hafta 9'da doğru biçimde yazmayı öğreneceğiz. `String` karşılaştırırken `==` yerine `equals()` kullanma alışkanlığını şimdiden edinin.

> **✓ İPUCU**

Buradaki kopya **sığdır** (shallow copy): alanlar tek tek kopyalanır, ancak bir alan başka bir nesneye referans veriyorsa o nesne kopyalanmaz, adresi paylaşılır. `Ogrenci` sınıfında yalnızca `String` ve sayısal alanlar olduğu için sorun çıkmaz; `String` değişmezdir (immutable).

4.6. toString() ile Anlamlı Metin Gösterimi

Bir nesneyi `System.out.println` ile yazdırdığınızda Java o nesnenin `toString()` metodunu çağırır. Bu metodu yazmazsanız, `Ogrenci@3f2a1b` gibi sınıf adı ve onaltılık bir sayıdan oluşan işe yaramaz bir metin görürsünüz.

Ogrenci.java

```java
/**
 * Nesnenin metin gosterimi. System.out.println(nesne) cagrildiginda
 * Java bu metodu otomatik olarak calistirir.
 */
@Override
public String toString() {
    return String.format("%d | %-14s | vize:%5.1f final:%5.1f ort:%6.2f | %s",
            numara, tamAd(), vizeNotu, finalNotu,
            ortalamaHesapla(), gectiMi() ? "GECTI" : "KALDI");
}
```

Kod 2.13 — `toString()` ezmesi — nesnenin okunabilir gösterimi

Öğe	Anlamı
`@Override`	Üst sınıftan gelen bir metodu ezdiğinizi derleyiciye bildirir; yazım hatası yaparsanız derleyici uyarır
`public String toString()`	İmza tam olarak böyle olmalıdır: `public`, `String` döner, parametresizdir
`String.format(...)`	`printf` ile aynı biçim belirteçlerini kullanır ama ekrana yazmaz, `String` döndürür
`%-14s`	Sola dayalı, en az 14 karakter genişliğinde metin
`%6.2f`	Toplam 6 karakter genişliğinde, virgülden sonra 2 basamaklı sayı
`%d`	Tam sayı (`int`, `long`)

Tablo 2.11 — `toString()` metodunun bileşenleri ve biçim belirteçleri

ⓘ BİLGİ

`toString()`, her Java sınıfının atası olan `Object` sınıfından gelir. Yazmadığınız zaman `Object` sürümü çalışır. Bu, dilin **çok biçimlilik** mekanizmasının ilk örneğidir; Hafta 9'da ayrıntısıyla ele alacağız.

✓ İPUCU

`toString()` içinde **hata fırlatmayın** ve **yan etki üretmeyin**. Bu metot hata ayıklayıcılar ve günlük (log) kayıtları tarafından beklenmedik anlarda çağrılır; içinde ağır iş yapmamalıdır.

4.7. Nesne Yaşam Döngüsü, Çöp Toplayıcı ve Yıkıcı Sorunu

C dilinde `malloc` ile ayırdığınız belleği `free` ile geri vermek zorundaydınız. Unutursanız **bellek sızıntısı** (memory leak), iki kez çağırırsanız **çift serbest bırakma** (double free) hatası oluşurdu. C++'ta bu iş yıkıcı (destructor) metoda bırakılmıştı.

Java'da **yıkıcı yoktur.** Belleği geri alma işi **çöp toplayıcıya** (garbage collector, GC) aittir. GC, artık hiçbir referans tarafından gösterilmeyen nesneleri tespit eder ve belleklerini geri alır. Ne zaman çalışacağına JVM karar verir; programcı bunu **garanti edemez**.



Şekil 2.8 — Nesnenin doğuşundan çöp toplanmasına kadar geçtiği beş aşama. Son iki aşama programcının denetiminde değildir.

YasamDongusu.java

```
1      public static void main(String[] args) {
2
3          System.out.println("1) Nesne dogar (new) ve referansla erisilir:");
4          Ogrenci o = new Ogrenci("Ali", "Yilmaz", 331001, 65, 78);
5          System.out.println("   " + o);
6
7          System.out.println("\n2) Referans baska nesneye yonlendirilir:");
8          o = new Ogrenci("Ayse", "Demir", 331002, 70, 80);
9          System.out.println("   " + o);
10         System.out.println("   Ilk nesneye artik ERISILEMEZ -> cop toplayici adayi.");
11
12         System.out.println("\n3) Referans null yapilir:");
13         o = null;
14         System.out.println("   Ikinci nesne de erisilemez duruma geldi.");
15
16         System.out.println("\n4) Cop toplayici ONERISI (garanti degil):");
17         System.gc();
18         System.out.println("   System.gc() yalnizca bir ONERIDIR; JVM uymayabilir.");
```

Kod 2.14 — Referansın kaybolması ve `System.gc()` önerisi

```
▸ TERMINAL — YasamDongusu çıktısı (1–4. adımlar)
1) Nesne dogar (new) ve referansla erisilir:
   331001 | Ali Yilmaz     | vize: 65.0 final: 78.0 ort: 72.80 | GECTI

2) Referans baska nesneye yonlendirilir:
   331002 | Ayse Demir     | vize: 70.0 final: 80.0 ort: 76.00 | GECTI
```

```
   Ilk nesneye artik ERISILEMEZ -> cop toplayici adayi.

3) Referans null yapilir:
   Ikinci nesne de erisilemez duruma geldi.

4) Cop toplayici ONERISI (garanti degil):
   System.gc() yalnizca bir ONERIDIR; JVM uymayabilir.
```

Soru	C	C++	Java
Bellek nasıl ayrılır?	`malloc(sizeof(T))`	`new T()`	`new T()`
Bellek nasıl geri verilir?	`free(p)` — elle	`delete p` — elle	**Çöp toplayıcı** — otomatik
Yıkıcı var mı?	Yok	Var (`~T()`)	**Yok**
Ne zaman geri alınır?	`free` çağrıldığı an	`delete` çağrıldığı an	**Belirsiz** — JVM'e bağlı
Sızıntı olur mu?	Evet (unutulursa)	Evet (unutulursa)	Nadiren (gereksiz referans tutulursa)
Kaynak (dosya) kapatma	`fclose` — elle	Yıkıcıda	`try-with-resources`

Tablo 2.12 — Bellek yönetiminin C, C++ ve Java'daki karşılaştırması

> **▲ DİKKAT**
>
> `System.gc()` çöp toplayıcıyı **çalıştırmaz**; yalnızca bir *öneride* bulunur. JVM bu öneriyi tamamen yok sayabilir. Üretim kodunda `System.gc()` çağırmak neredeyse her zaman bir tasarım hatasının belirtisidir.

Peki dosya, veritabanı bağlantısı, ağ soketi gibi kaynaklar ne olacak? Bunların **zamanında** kapatılması gerekir; GC'nin keyfini bekleyemeyiz. Java'nın buna verdiği yanıt `AutoCloseable` arayüzü ve `try-with-resources` yapısıdır.

YasamDongusu.java

```java
/** Yonetilmesi gereken bir kaynagi temsil eder. */
static class Kaynak implements AutoCloseable {
    private final String ad;

    Kaynak(String ad) {
        this.ad = ad;
        System.out.println("  [ACILDI]  " + ad);
    }

    void kullan() {
        System.out.println("  [KULLANIM] " + ad);
    }

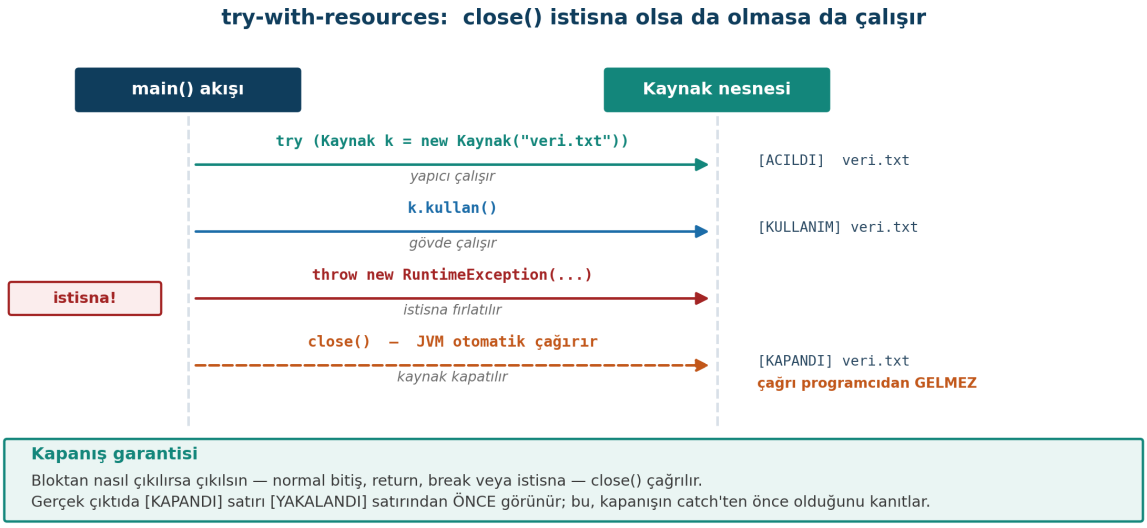
    @Override
    public void close() {
        System.out.println("  [KAPANDI] " + ad);
    }
}
```

Kod 2.15 — `AutoCloseable` gerçekleyen bir kaynak sınıfı

```java
// YasamDongusu.java
        System.out.println("\n5) Kaynak yonetimi: try-with-resources");
        try (Kaynak k = new Kaynak("veri.txt")) {
            k.kullan();
        }   // close() burada OTOMATIK cagrilir
        System.out.println("   try blogu bitti; close() calisti.");

        System.out.println("\n6) Hata olsa bile close() calisir:");
        try (Kaynak k = new Kaynak("ayarlar.txt")) {
            k.kullan();
            if (k != null) {
                throw new RuntimeException("beklenmedik hata");
            }
        } catch (RuntimeException e) {
            System.out.println("   [YAKALANDI] " + e.getMessage());
        }
```

Kod 2.16 — `try-with-resources`: normal bitişte de istisnada da `close()` çalışır



*Şekil 2.9 — `try` bloğundan nasıl çıkılırsa çıkılsın `close()` çağrılır; üstelik `catch` bloğundan **önce**.*

```
▸ TERMINAL — YasamDongusu çıktısı (5–6. adımlar)
5) Kaynak yonetimi: try-with-resources
  [ACILDI]  veri.txt
  [KULLANIM] veri.txt
  [KAPANDI] veri.txt
   try blogu bitti; close() calisti.

6) Hata olsa bile close() calisir:
  [ACILDI]  ayarlar.txt
  [KULLANIM] ayarlar.txt
  [KAPANDI] ayarlar.txt
   [YAKALANDI] beklenmedik hata
```

Çıktıdaki sıralamayı dikkatle okuyun: `[KAPANDI] ayarlar.txt` satırı `[YAKALANDI] beklenmedik hata` satırından **önce** yazılmıştır. Bu, kaynağın istisna yakalanmadan önce kapatıldığını kanıtlar. `finally` ile

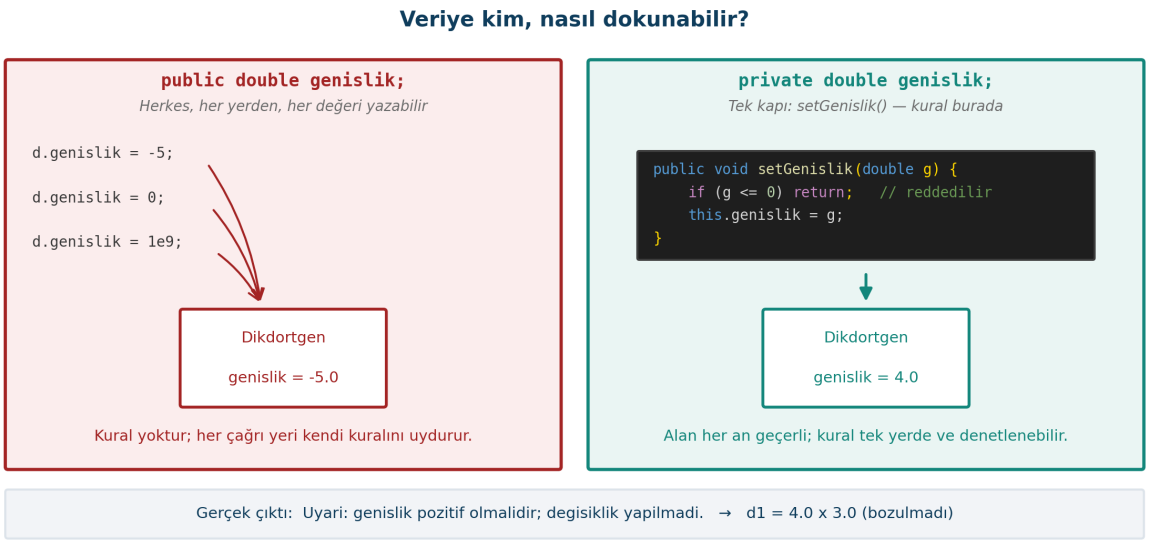
elle yazılan kapatma kodu da aynı işi görür, ancak `try-with-resources` daha kısa ve unutulmaya kapalıdır. İstisna yönetimini Hafta 11'de ayrıntılı göreceğiz.

> **ⓘ BİLGİ**
>
> `finalize()` metodu bir zamanlar "Java'nın yıkıcısı" diye anlatılırdı. Java 9'da **kullanımdan kaldırıldı** (deprecated), Java 18'de silinmeye aday gösterildi. Ne zaman çalışacağı, hatta çalışıp çalışmayacağı belirsiz olduğu için hiçbir zaman güvenilir bir araç olmadı. **Kullanmayın.**

4.8. Kapsüllemeye İlk Adım: private Alan ve Denetimli Erişim

Bölüm 2'de gördüğümüz üçüncü sorun (geçersiz değer atanabilmesi) yapıcı sayesinde **doğum anında** çözüldü. Ancak nesne doğduktan sonra da bozulabilir: `d.genislik = -5;` satırı alan `public` olduğu sürece serbesttir.

> **Kapsülleme (encapsulation)** Bir nesnenin iç verisini dış dünyadan gizleyip erişimi yalnızca sınıfın kendi metotları üzerinden vermektir. Böylece veri üzerindeki kurallar tek bir yerde toplanır ve her zaman uygulanır.



Şekil 2.10 — `public` alanda her çağrı yeri kendi kuralını uydurur; `private` alanda kural tek kapıda, setter'da tanımlıdır.

Dikdortgen.java

```
1  public class Dikdortgen {
2
3      private double genislik;
4      private double yukseklik;
5
6      public Dikdortgen(double genislik, double yukseklik) {
7          this.genislik = pozitifYap(genislik);
8          this.yukseklik = pozitifYap(yukseklik);
9      }
10
11     /** Kare olusturmak icin kisa yapici. */
```

```
    public Dikdortgen(double kenar) {
        this(kenar, kenar);
    }

    private static double pozitifYap(double deger) {
        return (deger > 0) ? deger : 1.0;
    }
```

Kod 2.17 — `private` alanlar ve doğrulayan yapıcılar

● *Dikdortgen.java*

```
    // --- Okuma erisimi (getter) ---
    public double getGenislik() {
        return genislik;
    }

    public double getYukseklik() {
        return yukseklik;
    }

    // --- Denetimli yazma erisimi (setter) ---
    public void setGenislik(double genislik) {
        if (genislik <= 0) {
            System.out.println("Uyari: genislik pozitif olmalidir; degisiklik yapilmadi.");
            return;
        }
        this.genislik = genislik;
    }

    public void setYukseklik(double yukseklik) {
        if (yukseklik <= 0) {
            System.out.println("Uyari: yukseklik pozitif olmalidir; degisiklik
yapilmadi.");
            return;
        }
        this.yukseklik = yukseklik;
    }
```

Kod 2.18 — Getter ve denetimli setter metotları

▸ TERMINAL — Dikdortgen çıktısı

```
d1   = 4.0 x 3.0 | alan=12.00 | cevre=14.00 | dikdortgen
kare = 5.0 x 5.0 | alan=25.00 | cevre=20.00 | kare

--- Gecersiz deger denemeleri ---
new Dikdortgen(-10, 0) -> 1.0 x 1.0 | alan=1.00 | cevre=4.00 | kare
Uyari: genislik pozitif olmalidir; degisiklik yapilmadi.
d1 (setGenislik(-5) sonrasi) = 4.0 x 3.0 | alan=12.00 | cevre=14.00 | dikdortgen
```

Erişim belirleyicisi	Aynı sınıf	Aynı paket	Alt sınıf	Herkes
`private`	Evet	Hayır	Hayır	Hayır
(belirtilmemiş — paket)	Evet	Evet	Hayır	Hayır
`protected`	Evet	Evet	Evet	Hayır

Erişim belirleyicisi	Aynı sınıf	Aynı paket	Alt sınıf	Herkes
`public`	Evet	Evet	Evet	Evet

Tablo 2.13 — Dört erişim düzeyi ve görünürlük alanları

Bu hafta yalnızca `private` ve `public` ile çalışacağız. `protected` kalıtımla birlikte anlam kazanır (Hafta 7); paket düzeyi erişim ise paket kavramını gördüğümüz Hafta 4'te ele alınacaktır.

Kural	Doğru	Yanlış
Getter adı	`getGenislik()`	`Genislik()`, `genislik()`
`boolean` getter	`isKare()` veya `karedirMi()`	`getKare()`
Setter adı	`setGenislik(double g)`	`genislikAyarla(double g)`
Setter dönüşü	`void` (veya akıcı stil için `this`)	Alanın kendisi
Doğrulama yeri	Setter'ın içinde	Çağrı yerinde, her seferinde

Tablo 2.14 — Getter/setter adlandırma sözleşmesi (JavaBeans kuralı)

▲ DİKKAT

Her `private` alan için otomatik olarak getter ve setter yazmak **kapsülleme değildir**; yalnızca alanı iki metotla sarmalamaktır. Gerçek kapsülleme, dışarının bilmesi gerekmeyen alanlar için **hiç** erişimci yazmamak ve setter'lara gerçek kurallar koymaktır. Örneğin `Dikdortgen` sınıfında `alan` bir alan değil, hesaplanan bir metottur — böylece tutarsızlık imkânsızdır.

5. C'den Java'ya Köprü

⇄ C DİLİNDEN JAVA'YA KÖPRÜ

C'de bir `struct` tanımladıktan sonra onu doldurmak için genellikle bir `hesap_ilklendir(&h, ...)` fonksiyonu yazardınız. Bu, tamamen sizin disiplininize bağlı bir sözleşmeydi: fonksiyonu çağırmayı unutan kimseyi derleyici uyarmazdı. Java'nın yapıcısı, bu sözleşmeyi **dilin zorunlu kuralı** hâline getirir.

- C: `struct Hesap h; hesap_ilklendir(&h, "TR1", 100);` — iki adım, ikincisi unutulabilir
- Java: `Hesap h = new Hesap("TR1", 100);` — tek adım, atlanamaz
- C: `h.bakiye = -500;` her yerden serbest
- Java: `private double bakiye;` — dışarıdan erişilemez

Kavram	C dilinde	Java'da
Veri kalıbı	`struct Ogrenci { ... };`	`class Ogrenci { ... }`
Kalıptan üretim	`struct Ogrenci o;` (yığında)	`Ogrenci o = new Ogrenci(...);` (öbekte)
İlklendirme	Elle yazılan `ogrenci_init(&o, ...)` fonksiyonu	**Yapıcı** — `new` ile otomatik çağrılır
İlklendirme zorunlu mu?	Hayır, unutulabilir	Evet, uygun yapıcı yoksa derlenmez
Kendine referans	Fonksiyona geçirilen `Ogrenci *this` işaretçisi	Örtük `this` referansı
Alan erişimi	`o.ad` veya `p->ad`	`o.getAd()` (alan `private` ise)
Veri gizleme	Yok (yalnızca `static` ile dosya kapsamı)	`private`, `protected`, `public`
Aynı adlı iki fonksiyon	Yasak — derleme hatası	Serbest — **aşırı yükleme**
Bellek geri verme	`free(p)`	Çöp toplayıcı
Metin gösterimi	Elle yazılan `ogrenci_yazdir(&o)`	`toString()` ezmesi
Kaynak kapatma	`fclose(f)` — elle	`try-with-resources` — otomatik

Tablo 2.15 — C ve Java arasındaki kavram karşılıkları

Aşağıdaki iki kod parçası **aynı işi** yapar. Soldaki C alışkanlığı, sağdaki Java karşılığıdır. Java sürümünde nesnenin geçersiz bir ara durumda hiç bulunmadığına dikkat edin.

```c
/* C yaklasimi: iki adim, ikincisi unutulabilir */
struct Ogrenci o;
ogrenci_init(&o, "Ali", "Yilmaz", 331001);
/* init cagrilmazsa o icinde cop deger bulunur */
```

Kod 2.19 — C: kalıp ve ilklendirme ayrı adımlardır

```java
// Java yaklasimi: tek adim, atlanamaz
Ogrenci o = new Ogrenci("Ali", "Yilmaz", 331001);
// Uygun yapici yoksa bu satir DERLENMEZ
```

Kod 2.20 — Java: üretim ve ilklendirme tek işlemdir

> ⓘ **BİLGİ**
>
> C bilenler için en şaşırtıcı ayrıntı, `new` ile ayrılan belleğin **çöp değer içermemesidir**. Java, nesne alanlarını yapıcı çalışmadan önce varsayılan değerlere kurar (`0`, `false`, `null`). Bu nedenle Java'da "ilklendirilmemiş alan okuma" diye bir hata sınıfı yoktur.

6. Sık Yapılan Hatalar

6.1. Yapıcıya Dönüş Türü Yazmak

> ✕ **SIK YAPILAN HATA**
>
> Yapıcının önüne `void` yazmak onu sıradan bir metoda dönüştürür. Derleyici tanım satırında hata vermez; hata `new` çağrısında ortaya çıkar ve iletisi yanıltıcıdır.

```
// YANLIS
public void Ogrenci(String ad) {   // <-- void yazildi: bu bir METOTTUR
    this.ad = ad;
}
// new Ogrenci("Ali");  ->  constructor Ogrenci in class Ogrenci
//                          cannot be applied to given types

// DOGRU
public Ogrenci(String ad) {        // donus turu YOK
    this.ad = ad;
}
```

Kod 2.21 — Dönüş türü yazılan yapıcı sıradan metoda dönüşür

6.2. this'i Unutmak

> ✕ **SIK YAPILAN HATA**
>
> Parametre adı alan adıyla aynıysa ve `this` yazılmazsa, atama parametreyi kendisine yapar. **Derleyici hata vermez**; alan `null` ya da `0` kalır. Bu, en zor fark edilen hatalardan biridir.

```
// YANLIS                          // DOGRU
public Ogrenci(String ad) {        public Ogrenci(String ad) {
    ad = ad;   // hicbir sey          this.ad = ad;
}                                  }
// Sonuc: ad alani null kalir      // Sonuc: ad alani dolar
```

Kod 2.22 — `this` unutulduğunda alan hiç değişmez

> ✓ **İPUCU**

Bu hatayı yakalamanın pratik yolu: nesneyi ürettikten hemen sonra `System.out.println(nesne)` yazın. `toString()` doğru yazılmışsa `null` veya `0.0` değerleri hemen göze çarpar.

6.3. Parametreli Yapıcı Ekleyip new Sinif() Kullanmaya Devam Etmek

✕ SIK YAPILAN HATA

Sınıfa parametreli bir yapıcı eklediğiniz anda varsayılan yapıcı kaybolur. Daha önce çalışan `new Sinif()` satırları artık derlenmez.

```
class Urun {
    private String ad;
    public Urun(String ad) { this.ad = ad; }   // varsayilan KAYBOLDU
}

Urun u = new Urun();     // DERLENMEZ
// Cozum: parametresiz yapiciyi acikca yaz ve zincirle
// public Urun() { this("Isimsiz"); }
```

Kod 2.23 — Varsayılan yapıcının kaybolması

6.4. this(...) Çağrısını İlk Deyim Yapmamak

✕ SIK YAPILAN HATA

`this(...)` çağrısı yapıcının **ilk** deyimi olmak zorundadır. Önüne bir günlük satırı bile yazamazsınız.

```
// YANLIS
public Nokta() {
    System.out.println("olusuyor");   // <-- once bu var
    this(0.0, 0.0);                   // call to this must be first statement
}

// DOGRU
public Nokta() {
    this(0.0, 0.0);
    System.out.println("olustu");
}
```

Kod 2.24 — `this(...)` ilk deyim olmak zorundadır

6.5. Kopya Sanılan Atama

✕ SIK YAPILAN HATA

`Ogrenci kopya = ozgun;` satırı kopya **üretmez**; yalnızca ikinci bir referans oluşturur. Birini değiştirmek diğerini de değiştirir.

```
Ogrenci kopya = ozgun;              // TEK nesne, iki referans
kopya.notlariGuncelle(95, 95);      // ozgun de degisti!
System.out.println(ozgun == kopya); // true

Ogrenci kopya2 = new Ogrenci(ozgun);   // IKI nesne
System.out.println(ozgun == kopya2);   // false
```

Kod 2.25 — Atama ile kopya yapıcı arasındaki fark

6.6. Derleyici İletileri Sözlüğü

Derleyici iletisi	Gerçek nedeni	Çözüm
`constructor X in class X cannot be applied to given types`	Verdiğiniz argümanlara uyan yapıcı yok (çoğu zaman varsayılan yapıcı kaybolmuştur)	Uygun yapıcıyı yazın veya çağrıdaki argümanları düzeltin
`call to this must be first statement in constructor`	`this(...)` ilk deyim değil	`this(...)` satırını yapıcının en başına taşıyın
`recursive constructor invocation`	Yapıcı zinciri döngüsel: A→B ve B→A	Zincirin ucunda gerçek işi yapan bir yapıcı bırakın
`X has private access in Y`	`private` alana sınıf dışından erişilmeye çalışıldı	Getter/setter kullanın veya erişimi sınıf içine taşıyın
`invalid method declaration; return type required`	Sınıf adından farklı bir ada sahip metoda dönüş türü yazılmamış (genellikle yapıcı adı yanlış yazılmıştır)	Yapıcının adını sınıf adıyla birebir eşleyin
`variable x might not have been initialized`	Yerel değişken atanmadan kullanıldı (alanlarda bu hata olmaz)	Değişkene tanımlandığı yerde değer verin
`non-static variable this cannot be referenced from a static context`	`static` metot içinde `this` kullanıldı	`this` yerine nesne üzerinden erişin veya metodu `static` olmaktan çıkarın

Tablo 2.16 — Bu haftaya özgü derleyici iletileri ve çözümleri

7. İyi Tasarım İlkeleri

Aşağıdaki ilkeler bu haftanın konusuna özgü, sınavda ve laboratuvar değerlendirmesinde aranan yazım standartlarıdır.

1. **Nesneyi geçerli doğurun.** Yapıcı bittiğinde nesne, her metodu güvenle çağrılabilecek durumda olmalıdır. “Sonra doldururum” yaklaşımından kaçının.
2. **Gerçek işi tek bir yapıcıda toplayın.** Diğer yapıcılar `this(...)` ile ona yönlensin. Doğrulama kuralı böylece tek yerde bulunur.
3. **Doğrulamayı yapıcıya ve setter'a koyun.** İş kuralı sınıfın içinde yaşamalı, çağrı yerlerine dağılmamalıdır.
4. **Alanları `private` yapın.** Dışarının okuması gerekiyorsa getter yazın; yazması gerekmiyorsa setter **yazmayın**.
5. **Hesaplanabilen değeri alan olarak saklamayın.** `Dikdortgen` sınıfında `alan` bir alan değil, `alan()` metodudur; böylece `genislik` değiştiğinde tutarsızlık oluşamaz.
6. **Her sınıfa `toString()` yazın.** Hata ayıklama süresini belirgin biçimde kısaltır; `@Override` etiketini unutmayın.
7. **Parametrelere alan adlarını verin ve `this` kullanın.** Bu, Java topluluğunun yerleşik alışkanlığıdır.
8. **Yapıcıda ağır iş yapmayın.** Dosya okuma, ağ bağlantısı gibi işler yapıcıda değil, ayrı bir metotta yapılmalıdır; yarım kalan bir yapıcı nesneyi kullanılamaz bırakır.
9. **Kaynak tutan sınıflar `AutoCloseable` gerçeklesin.** Kapatmayı `finalize` veya çöp toplayıcıya bırakmayın; `try-with-resources` kullanın.
10. **Parametresiz yapıcıya ihtiyaç varsa onu açıkça yazın.** Varsayılan yapıcının sessizce kaybolmasına güvenmeyin.

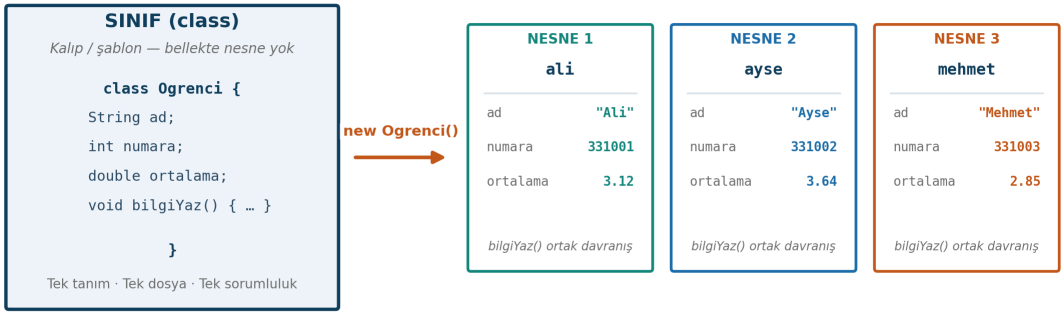
Durum	Önerilen yaklaşım
Sınıfın tüm bilgileri her zaman biliniyor	Tek bir tam yapıcı yeter
Bazı bilgiler sonradan girilebilir	Kısa yapıcı + `this(...)` zinciri + denetimli setter
Nesnenin bağımsız bir kopyası gerekiyor	Kopya yapıcı yazın
Üretim mantığı karmaşık, ada ihtiyaç var	Statik fabrika metodu (Hafta 5 ve 14)
Alan üretimden sonra hiç değişmemeli	`private final` alan, setter yazmayın
Nesne bir kaynak tutuyor	`AutoCloseable` + `try-with-resources`

Tablo 2.17 — Hangi durumda hangi üretim düzeni seçilmeli?

8. Bir Sonraki Haftaya Köprü

Bu hafta bir sınıfın nesnelerini **güvenli biçimde üretmeyi** öğrendik. Kapsüllemenin ilk adımını attık: alanları `private` yaptık, erişimi metotlara bıraktık. Ancak kapsülleme, nesne yöneliminin **dört temel ilkesinden yalnızca biridir**.

Hafta 3'te bu dört ilkeyi — kapsülleme, soyutlama, miras ve çok biçimlilik — tek bir örnek sistem üzerinde uçtan uca göreceğiz. Amaç derinlik değil **bütünlük** olacak: her ilkenin hangi problemi çözdüğünü ve birbirleriyle nasıl ilişkilendiğini kavrayacağız. Ayrıntılar, tabloda belirtilen haftalara bırakılacaktır.



Sınıf birdir; nesne sayısı sınırsızdır. Her nesnenin kendi alan değerleri vardır, davranış tanımı ortaktır.

Şekil 2.11 — Hafta 1'den hatırlatma: sınıf kalıptır, nesne o kalıptan üretilen örnektir. Hafta 3'te bu resmin üzerine dört ilkeyi yerleştireceğiz.

Bu hafta öğrendiğiniz	İlerideki karşılığı	Hafta
Yapıcı metot	`super(...)` ile üst sınıf yapıcısının çağrılması	7
`this(...)` zinciri	Nesne başlatma sırası ve başlatma blokları	5
`private` alan + getter/setter	Dört erişim düzeyi, `final`, immutable sınıf	4
`toString()` ezmesi	`equals()`, `hashCode()` ve çok biçimlilik	9
Aşırı yükleme	Derleme zamanı çok biçimliliği	9
`AutoCloseable` / `try-with-resources`	İstisna yönetimi ve kaynak güvenliği	11
`private static` yardımcı metot	`static` alanlar, sayaçlar, utility sınıflar	6
Kopya yapıcı	Statik fabrika metotları ve derin kopya	5 ve 14

Tablo 2.18 — Bu haftanın kavramlarının ileriki haftalardaki karşılıkları

9. Haftanın Özeti

- **Yapıcı metot**, nesne `new` ile doğarken otomatik çalışan ve ona geçerli bir başlangıç durumu veren özel bir bloktur. Adı sınıf adıyla aynıdır, dönüş türü yazılmaz ve yalnızca `new` ile çağrılır.
- Yapıcı sayesinde nesne **hiçbir an geçersiz durumda bulunmaz**; iş kuralı çağrı yerlerine dağılmak yerine sınıfın içinde tek yerde toplanır.
- **`this`**, üzerinde çalışılan nesnenin kendisine verilen referanstır. Alan gölgelemesini çözmek (`this.ad = ad;`), nesneyi döndürmek (`return this;`) ve başka bir yapıcıyı çağırmak (`this(...)`) için kullanılır.
- **Aşırı yükleme**, aynı sınıfta parametre listesi farklı birden çok yapıcı bulunmasıdır. Ayırt edici ölçüt yalnızca parametre sayısı, türü ve sırasıdır; parametre adı ve dönüş türü ayırt edici değildir.

- **`this(...)` zinciri** ile tüm yapıcılar tek bir tam yapıcıya yönlendirilir; kod tekrarı ortadan kalkar. `this(...)` yapıcının ilk deyimi olmak zorundadır.
- **Varsayılan yapıcı** yalnızca hiç yapıcı yazmadığınızda derleyici tarafından üretilir. Kendi yapıcınızı yazdığınız anda kaybolur; hâlâ gerekiyorsa açıkça yazmalısınız.
- **Kopya yapıcı** bağımsız bir nesne üretir; `kopya = ozgun;` ise yalnızca ikinci bir referans oluşturur. Farkı `==` ile sınayabilirsiniz.
- Java'da **yıkıcı yoktur**; belleği çöp toplayıcı geri alır ve ne zaman çalışacağı belirsizdir. `System.gc()` yalnızca bir öneridir, `finalize()` ise kullanımdan kaldırılmıştır.
- Dosya, bağlantı gibi kaynaklar için doğru araç **`AutoCloseable` + `try-with-resources`**'tır; `close()` bloktan nasıl çıkılırsa çıkılsın çalışır.
- Alanları **`private`** yapıp erişimi getter/setter ile vermek kapsüllemenin ilk adımıdır. Doğrulama mantığı setter'ın içinde bulunmalıdır.

BİR SONRAKİ HAFTA

Hafta 3'te nesne yöneliminin dört temel ilkesini — kapsülleme, soyutlama, miras ve çok biçimlilik — tek bir örnek sistem üzerinde bütünsel olarak göreceğiz. Bu hafta yazdığınız `Ogrenci` ve `Dikdortgen` sınıfları o örneğin başlangıç noktası olacak. Laboratuvar öncesinde `Kitap` sınıfınızı elinizin altında bulundurun.

10. Alıştırmalar

Alıştırmalar üç gruba ayrılmıştır: **kavram soruları** (1–4), **kod okuma ve hata bulma** (5–7) ve **programlama** (8–12). Zorluk düzeyleri: ●○○ temel, ●●○ orta, ●●● ileri.

10.1. Kavram Soruları

Alıştırma 1 ●○○

Yapıcı metodu sıradan bir metottan ayıran **üç** özelliği yazın. Her biri için, o özelliğin ihlal edildiğinde ortaya çıkan sonucu bir cümleyle açıklayın.

Alıştırma 2 ●○○

Bir sınıfta yalnızca `public Urun(String ad, double fiyat)` yapıcısı tanımlı. `new Urun()` satırı neden derlenmez? Derleyici hangi iletiyi verir? Bu satırın çalışması için sınıfa ne eklenmelidir? Eklenecek kodu `this(...)` zinciriyle yazın.

Alıştırma 3 ●●○

`this` anahtar sözcüğünün dört kullanım biçimini örnek kod satırlarıyla gösterin. Ardından şu soruyu yanıtlayın: `static` bir metot içinde `this` neden kullanılamaz?

Alıştırma 4 ●●○

Java'da yıkıcı (destructor) bulunmamasının nedenini açıklayın. C++'taki `~Sinif()` yerine Java hangi iki mekanizmayı sunar? `finalize()` metodunun neden kullanımdan kaldırıldığını gerekçesiyle yazın.

İpucu: Bölüm 4.7'deki karşılaştırma tablosunu inceleyin.

10.2. Kod Okuma ve Hata Bulma

Alıştırma 5 ●●○

Aşağıdaki sınıfta **üç** hata vardır. Her birini bulun, neden hata olduğunu açıklayın ve düzeltilmiş sürümü yazın.

```
class Kalem { private String renk; private double fiyat; public void Kalem(String renk) { renk = renk; } public Kalem(String renk, double fiyat) { System.out.println("olustu"); this(renk); this.fiyat = fiyat; } }
```

İpucu: Bölüm 6.1, 6.2 ve 6.4'e bakın.

Alıştırma 6 ●●○

Aşağıdaki kod parçası ekrana ne yazar? Yanıtınızı, nesne sayısını ve referansların hangi nesneyi gösterdiğini

çizerek gerekçelendirin.

```
Nokta a = new Nokta(2.0, 5.0); Nokta b = a; Nokta c = new Nokta(a); b.tasi(1, 1);
System.out.println(a + " " + b + " " + c); System.out.println((a == b) + " " + (a == c));
```

Alıştırma 7 ●○○

Bir öğrenci, `Ogrenci` sınıfının yapıcısını şöyle yazmış: `public Ogrenci(String ad, int numara) { ad = ad; numara = numara; }` Program derleniyor ve çalışıyor; ancak `toString()` her zaman `null` ve `0` yazıyor. Nedenini açıklayın ve **iki** farklı çözüm önerin.

10.3. Programlama Alıştırmaları

Alıştırma 8 ●○○

`Sicaklik` adında bir sınıf yazın. Alan: `private double celsius`. İki yapıcı bulunsun: parametresiz (0 °C) ve `double` alan. Sıcaklık mutlak sıfırın (−273.15 °C) altındaysa yapıcı değeri −273.15'e çeksin. `fahrenheit()` ve `kelvin()` metotlarını ve `toString()` metodunu yazın. Sınıfı en az beş farklı değerle sınayın.

Alıştırma 9 ●●○

`Dikdortgen` sınıfını örnek alarak bir `Daire` sınıfı yazın. `private double yaricap` alanı bulunsun; yapıcı negatif yarıçapı reddedip `1.0` yapsın. `alan()`, `cevre()`, `capiniHesapla()` metotlarını ve denetimli bir `setYaricap` setter'ını yazın. `toString()` çıktısı `Daire r=2.50 alan=19.63 cevre=15.71` biçiminde olsun.

İpucu: `Math.PI` sabitini ve `String.format` ile `%.2f` biçim belirtecini kullanın.

Alıştırma 10 ●●●

`Kesir` sınıfı yazın. Alanlar: `private int pay, payda`. Üç yapıcı olsun: `Kesir()` → 0/1, `Kesir(int pay)` → pay/1, `Kesir(int pay, int payda)`. Payda sıfır verilirse `1` kabul edilsin. Yapıcı kesri **sadeleştirsin** (EBOB ile) ve eksi işaretini paya taşısın. `topla`, `carp` metotları yeni bir `Kesir` **döndürsün** (var olanı değiştirmesin). `toString()` `3/4` biçiminde yazsın.

İpucu: EBOB için Öklid algoritmasını `private static int ebob(int a, int b)` biçiminde yazın. Yapıcıları `this(...)` ile zincirleyin.

Alıştırma 11 ●●●

`Ogrenci` sınıfına bir **kopya yapıcı** ve `notlariGuncelle` metodu zaten var. Buna ek olarak, verilen bir `Ogrenci[]` dizisinin **derin kopyasını** üreten `public static Ogrenci[] kopyala(Ogrenci[] kaynak)` metodunu yazın. Kopyadaki bir öğrencinin notunu değiştirdiğinizde özgün dizinin etkilenmediğini bir programla kanıtlayın.

İpucu: Diziyi kopyalamak yetmez; her elemanı da kopya yapıcıyla yeniden üretmelisiniz.

Alıştırma 12 ●●●

`GeciciDosya` adında, `AutoCloseable` arayüzünü gerçekleyen bir sınıf yazın. Yapıcısı ekrana `[ACILDI] <ad>`, `close()` metodu `[KAPANDI] <ad>` yazsın. Bir `try-with-resources` bloğu içinde iki nesne üretip, blok içinde bilerek bir `RuntimeException` fırlatın. Çıktıdaki kapanış **sırasını** gözlemleyin ve neden o sırada olduğunu açıklayın.

İpucu: Birden çok kaynak açıldığında `close()` çağrıları **ters** sırada yapılır.

11. Öz Değerlendirme

Aşağıdaki maddelerin tamamını işaretleyebiliyorsanız bu haftanın kazanımlarına ulaşmışsınız demektir. İşaretleyemediğiniz her madde için ilgili bölüme geri dönün.

Öz Değerlendirme Kontrol Listesi

- ☐ Yapıcı metodun üç ayırt edici özelliğini (ad, dönüş türü, çağrılma biçimi) sayabiliyorum.
- ☐ Bir sınıfa parametreli yapıcı yazıp nesneyi tek satırda geçerli durumda üretebiliyorum.
- ☐ `this.alan = parametre;` yazmamın neden zorunlu olduğunu açıklayabiliyorum.
- ☐ Bir sınıfa birden çok yapıcı yazıp derleyicinin hangisini neye göre seçtiğini anlatabiliyorum.
- ☐ `this(...)` ile yapıcı zinciri kurabiliyor ve ilk deyim kuralını biliyorum.
- ☐ Varsayılan yapıcının ne zaman kaybolduğunu ve buna karşılık gelen derleyici iletisini tanıyorum.
- ☐ Kopya yapıcı yazabiliyor ve `==` ile bağımsızlığı sınayabiliyorum.
- ☐ `toString()` metodunu `@Override` ile ezip biçimli çıktı üretebiliyorum.
- ☐ Nesne yaşam döngüsünün beş aşamasını sırayla anlatabiliyorum ve Java'da yıkıcı bulunmamasının nedenini biliyorum.
- ☐ `private` alan + getter/setter düzenini kurabiliyor ve doğrulamayı setter'a koyabiliyorum.
- ☐ `try-with-resources` yapısının `close()` çağrısını neden garanti ettiğini açıklayabiliyorum.

12. Kaynaklar

12.1. Ders Kitapları

Kaynak	İlgili bölüm	Odak
Deitel & Deitel, *Java How to Program*, 11. baskı	Bölüm 3 ve 8	Sınıf tanımı, yapıcılar, `this`, kapsülleme
Cay Horstmann, *Core Java Volume I*, 12. baskı	Bölüm 4.3–4.6	Yapıcılar, alan başlatma, nesne yıkımı
Herbert Schildt, *Java: The Complete Reference*, 12. baskı	Bölüm 6 ve 7	Yapıcı aşırı yükleme, `this`, çöp toplama
Joshua Bloch, *Effective Java*, 3. baskı	Madde 1, 2, 8, 9	Statik fabrika, `finalize` kaçınma, `try-with-resources`
Oracle, *The Java Tutorials*	"Classes and Objects" bölümü	Resmî başvuru kaynağı

Tablo 2.19 — Bu haftanın konusuyla ilgili okuma listesi

12.2. Çevrimiçi Kaynaklar

- Oracle Java Tutorials — *Providing Constructors for Your Classes*: `docs.oracle.com/javase/tutorial/java/javaOO/constructors.html`
- Oracle Java Tutorials — *The try-with-resources Statement*: `docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html`
- Java Language Specification, §8.8 *Constructor Declarations* — dilin resmî tanımı; kural tartışmalarında son sözü söyler.
- Oracle JDK 21 API belgeleri — `java.lang.Object` sınıfı (`toString`, `equals`, `finalize` açıklamaları).

12.3. Bu Haftanın Kod Dosyaları

Dosya	Konu	Bölüm
`OgrenciEski.java`	Hafta 1'den kalan yapıcısız sürüm	2.1
`SorunGosterimi.java`	Yapıcısız yaklaşımın üç sorunu	2.1
`Ogrenci.java`	Yapıcılı, `private` alanlı sürüm	3.2
`OgrenciDemo.java`	Yapıcı kullanımı ve kopya deneyi	3.3, 4.5
`Nokta.java`	Dört yapıcı, aşırı yükleme, zincirleme	4.2, 4.3
`NoktaDemo.java`	Yapıcı seçimi ve bağımsızlık sınaması	4.2
`GolgelemeOrnegi.java`	`this` ve alan gölgeleme	4.1
`VarsayilanYapici.java`	Varsayılan yapıcı, alan varsayılan değerleri	3.4, 4.4
`YasamDongusu.java`	Yaşam döngüsü, GC, `try-with-resources`	4.7
`Dikdortgen.java`	`private` alan, getter/setter, doğrulama	4.8

Tablo 2.20 — Ders örneklerinin dosya listesi (`Hafta_02/Java_Kodlari/ders/`)

12.4. İletişim

Rol	Ad	Görev
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Teori dersi, sınav, dönem projesi
Laboratuvar	Arş. Gör. Betül MUMCU	Laboratuvar oturumu, föy değerlendirmesi